

DocuMind Enterprise: Sistema Inteligente de Gestión y Análisis Documental

Fase 05: Implementación y despliegue

Juan Esteban Rojas Melendez

Universitaria Tecnológica de Santander

TABLA DE CONTENIDOS

1. Descripción del Ambiente de Implementación
2. Requisitos de Infraestructura (Hardware y Software)
3. Configuración de Base de Datos y Persistencia de Almacenamiento
4. Configuración Segura de Servicios de IA y Variables de Entorno
5. Proceso de Instalación y Despliegue de la Aplicación
6. Manual de Usuario Final
7. Manual de Administración y Soporte Técnico
8. Estrategia Básica de Respaldo y Recuperación ante Desastres
9. Plan Básico de Mantenimiento y Monitoreo
10. Evidencias del Sistema en Operación

1. Descripción del Ambiente de Implementación

La plataforma **DocuMind Enterprise** fue diseñada bajo un esquema de arquitectura desacoplada y contenerizada, lo que permite su puesta en marcha en servidores dedicados, infraestructura en la nube o entornos corporativos locales.

El ambiente de ejecución separa estrictamente tres niveles operativos:

- **Capa de Presentación (Frontend):** Servida de forma estática o a través de un servidor web ligero como Nginx / Node.js, interactuando de manera asíncrona vía HTTPS/REST.
- **Capa de Negocio (Backend):** Instancia de FastAPI gestionada mediante el servidor ASGI Uvicorn en un entorno de ejecución Python 3.10+ aislado.
- **Capa de Persistencia y Vectorización:** Contenedor de Docker con la imagen estandarizada de PostgreSQL 16 y la extensión **pgvector**, vinculada a un volumen lógico persistente que previene la pérdida de registros o índices tras reinicios.
- **Capa Externa de Inteligencia Artificial:** Conexión cifrada hacia la API de Google AI Studio (**gemini-3.6-flash** para tareas de PLN estructurado y **gemini-embedding-2** para representaciones vectoriales densas).

2. Requisitos de Infraestructura (Hardware y Software)

2.1 Requisitos Mínimos de Hardware (Servidor / Host)

- **Procesador (CPU):** 2 núcleos virtuales (vCPU) o físicos con arquitectura de 64 bits (x86_64).
- **Memoria RAM:** 4 GB de memoria física disponible (se recomiendan 8 GB para soportar la concurrencia en la fragmentación de documentos y Docker).
- **Almacenamiento en Disco:** Mínimo 10 GB de espacio libre en disco de estado sólido (SSD) para el volumen de Docker, el entorno virtual y la carpeta de persistencia documental **storage/**.
- **Conectividad de Red:** Conexión a Internet de banda ancha continua y con puertos de salida HTTPS habilitados para el consumo de Google Cloud AI.

2.2 Requisitos de Software y Dependencias

- **Sistema Operativo:** Windows 10/11 Pro (con WSL2 habilitado), Ubuntu Server 22.04 LTS o macOS con soporte Docker Desktop / Docker Engine.
- **Motor de Contenedores:** Docker Engine versión 24+ y Docker Compose v2+.
- **Python:** Intérprete nativo Python 3.10 o 3.11 instalado.
- **Node.js:** Versión 18 o 20 LTS junto con el gestor de paquetes **npm**.
- **Navegador Web:** Google Chrome, Mozilla Firefox, Microsoft Edge o navegadores compatibles con HTML5 y JavaScript ES6+.

3. Configuración de Base de Datos y Persistencia de Almacenamiento

La base de datos relacional y vectorial se aprovisiona mediante Docker Compose. Para asegurar la persistencia física entre ciclos de vida del contenedor, los archivos de la base de datos se ligan al volumen `documind_postgres_data`.

Archivo de Despliegue (`docker-compose.yml`)

```
YAML
version: '3.8'

services:
  documind_db:
    image: pgvector/pgvector:pg16
    container_name: documind_db
    restart: always
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: documind_password_secure
      POSTGRES_DB: documind_db
    ports:
      - "5432:5432"
    volumes:
      - documind_postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5

volumes:
  documind_postgres_data:
    name: documind_postgres_data
```

Una vez en marcha el contenedor, se debe ejecutar la inicialización de la extensión vectorial dentro del motor de PostgreSQL:

```
Bash
docker exec -i documind_db psql -U postgres -d documind_db -c
"CREATE EXTENSION IF NOT EXISTS vector;"
```

4. Configuración Segura de Servicios de IA y Variables de Entorno

En cumplimiento con el requerimiento no funcional de seguridad (RNF-01), todas las credenciales de conexión, llaves de API y configuraciones de tokens se segregan en el archivo `backend/.env`, el cual debe ser excluido estrictamente del sistema de control de versiones Git mediante `.gitignore`.

```
Plantilla del Archivo backend/.env
Fragmento de código
# Conexión a la Base de Datos Contenerizada
DATABASE_URL=postgresql://postgres:documind_password_secure@localhost:5432/documind_db

# Proveedor de Inteligencia Artificial (Google AI Studio)
GEMINI_API_KEY=AIzaSyA_TU_LLAVE_PRIVADA_AQUI
GEMINI_TEXT_MODEL=gemini-3.6-flash
GEMINI_EMBEDDING_MODEL=models/gemini-embedding-2

# Parámetros Criptográficos y de Sesión JWT
JWT_SECRET_KEY=clave_secreta_super_segura_uts_documind_enterprise_2026
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=120

# Parámetros del Sistema
MAX_UPLOAD_SIZE_MB=10
ALLOWED_EXTENSIONS=.pdf,.docx,.txt
STORAGE_PATH=storage/
```

5. Proceso de Instalación y Despliegue de la Aplicación

El despliegue de **DocuMind Enterprise** en una nueva infraestructura se realiza siguiendo un procedimiento secuencial de cinco pasos:

Paso 1: Clonación del Código y Preparación del Repositorio

```
Bash
git clone https://github.com/usuario/DocuMind-Enterprise.git
cd DocuMind-Enterprise
```

Paso 2: Despliegue del Contenedor de Base de Datos

```
Bash
docker compose up -d
docker exec -i documind_db psql -U postgres -d documind_db -c
"CREATE EXTENSION IF NOT EXISTS vector;"
```

Paso 3: Aprovisionamiento y Puesta en Marcha del Backend

```
Bash
cd backend
python -m venv venv

# Activación del entorno virtual
# En Windows:
.\venv\Scripts\activate
# En Linux/macOS:
# source venv/bin/activate

pip install --upgrade pip
pip install -r requirements.txt
python create_tables.py
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Paso 4: Generación Masiva de Datos de Prueba (Corpus de 30 Documentos)

En una consola secundaria con el entorno virtual activo:

```
Bash
cd backend
python generate_mock_data.py
```

Este comando puebla el directorio `backend/storage/` con 10 facturas en formato `.txt`, 10 contratos en formato `.docx` y 10 hojas de vida en formato `.pdf` con texto nativo indexable.

Paso 5: Compilación y Despliegue del Frontend

En una tercera consola:

```
Bash
cd ../frontend
npm install
npm run dev -- --host
```

La aplicación web estará disponible de inmediato en la dirección local <http://localhost:5173> y expuesta a la red de área local (LAN).

6. Manual de Usuario Final

6.1 Acceso e Inicio de Sesión (RF-01)

1. Abra su navegador web de preferencia y diríjase a la URL <http://localhost:5173>.
2. En la pantalla inicial de autenticación, ingrese su correo electrónico institucional (admin@documind.com) y su contraseña predeterminada ([admin123](#)).
3. Haga clic en **Iniciar Sesión**. El sistema validará sus credenciales, emitirá un token JWT seguro y le concederá acceso al panel principal.

6.2 Visualización del Dashboard (RF-17)

1. Acceda a la pestaña **Dashboard** situada en la barra de navegación lateral.
2. En la parte superior, revise los indicadores en tiempo real:
 - **Total Documentos:** Cantidad neta de archivos en el repositorio.
 - **Procesados Exitosamente:** Documentos leídos, clasificados e indexados con vectores.
 - **Errores de Extracción:** Documentos que no pudieron procesarse o superaron límites.
3. En la sección inferior, observe la tarjeta de distribución por tipologías documentales (Facturas, Contratos y Hojas de Vida).

6.3 Gestión del Repositorio y Carga de Archivos (RF-04, RF-05, RF-08)

1. Seleccione la opción **Repositorio** en la barra de navegación lateral.
2. Para cargar un nuevo archivo, haga clic en el botón azul **Cargar Documento**.
3. En la ventana del explorador, seleccione un archivo con formato admitido ([.pdf](#), [.docx](#) o [.txt](#)).
4. La interfaz notificará el estado *Procesando IA...* El backend guardará el binario, extraerá el texto, invocará al modelo [gemini-3.6-flash](#), generará los vectores con [gemini-embedding-2](#) y lo registrará en la tabla.
5. Para filtrar documentos en tiempo real (RF-15), utilice la barra de búsqueda superior por nombre o el menú desplegable por categoría.

6.4 Vista de Detalle, Resumen y Descarga (RF-06, RF-11 al RF-13)

1. En la tabla del repositorio, ubique el documento que desea consultar y presione el botón de inspección (icono de ojo o **Ver Detalle**).
2. Se desplegará un modal o panel dividido:

- **Izquierda (Metadatos e IA):** Muestra la tipología clasificada, el resumen ejecutivo generado en 2 a 4 oraciones y la estructura JSON limpia con las entidades extraídas.
- **Derecha (Acciones):** Muestra el botón **Descargar Archivo Original**, el cual descargará el binario desde </api/documents/{id}/download> directamente a su equipo.

6.5 Eliminación Sincronizada de Documentos (RF-07)

1. En la tabla de documentos, pulse sobre el icono de la papelera roja situado en la columna de acciones.
2. El navegador solicitará confirmación mediante un diálogo: *¿Desea eliminar el documento de forma permanente?*
3. Al aceptar, el sistema suprimirá el archivo físico de **storage/** y ejecutará un borrado en cascada en PostgreSQL eliminando sus metadatos y fragmentos vectoriales.

6.6 Asistente IA y Consultas RAG (RF-16)

1. Diríjase a la sección **Asistente IA** en la barra lateral.
2. En la caja de texto inferior, escriba una pregunta natural sobre el contenido de los documentos almacenados (por ejemplo: *"¿Cuáles son los valores y fechas de emisión de las facturas registradas?"* o *"¿Qué postulantes tienen experiencia en Python?"*).
3. Presione el botón **Enviar**.
4. El sistema calculará el embedding de su pregunta, recuperará los fragmentos relevantes de **pgvector** y el modelo responderá citando los nombres de los archivos fuente utilizados como soporte.
5. Si consulta por información inexistente, el asistente declarará explícitamente la ausencia de datos sin generar alucinaciones.

7. Manual de Administración y Soporte Técnico

7.1 Gestión de Servicios del Sistema

- **Revisar estado de la base de datos:**

```
Bash
docker ps
docker logs --tail 50 documind_db
```

- **Reinicio controlado del motor de datos:**

```
Bash
docker compose restart documind_db
```

- Acceso interactivo a la consola SQL:

```
Bash
docker exec -it documind_db psql -U postgres -d documind_db
```

7.2 Diagnóstico y Resolución de Incidentes Frecuentes

Síntoma Detectado	Causa Raíz Probable	Procedimiento de Resolución Técnica
El frontend muestra "Failed to fetch" al subir o eliminar archivos.	El servidor FastAPI (uvicorn) se detuvo o el entorno virtual no fue cargado.	Abrir la terminal de backend, activar el entorno virtual (.venv\Scripts\activate) e iniciar nuevamente Uvicorn en el puerto 8000.
El document o cambia a estado "Error" tras subirlo.	Exceso en la cuota gratuita de Google AI Studio (HTTP 429) o archivo sin texto legible.	Consultar la columna error_log en la base de datos o en la consola del backend. Esperar 60 segundos si fue por cuota y reintentar.
Error DataError : vector dimension	Discrepancia en la dimensión del embedding vector retornado por Gemini.	Verificar que en el backend se esté enviando el parámetro output_dimensionality=768 en las llamadas a models/gemini-embedding-2.

s do not match.		
Error Relation "documents" does not exist.	No se han ejecutado los scripts de migración DDL en la base de datos.	Ejecutar <code>python create_tables.py</code> en la terminal del backend para crear las tablas relacionales y vectoriales.

8. Estrategia Básica de Respaldo y Recuperación ante Desastres

8.1 Copia de Seguridad de la Base de Datos (Relacional + Vectores)

Se establece un procedimiento de respaldo lógico diario mediante la herramienta nativa `pg_dump`:

```
Bash
# Exportar respaldo completo con estructura y datos
docker exec -t documind_db pg_dump -U postgres -d documind_db >
backup_documind_$(date +%Y%m%d).sql
```

8.2 Copia de Seguridad del Repositorio Físico

Los archivos binarios originales se almacenan en el directorio `backend/storage/`. Se recomienda generar una instantánea comprimida periódica:

```
Bash
# Generar archivo comprimido del repositorio binario
tar -czvf storage_backup_$(date +%Y%m%d).tar.gz backend/storage/
```

8.3 Procedimiento de Restauración

En caso de fallo crítico en el servidor anfitrión:

1. Levantar un nuevo contenedor vacío de PostgreSQL mediante `docker compose up -d`.
2. Restaurar la base de datos ejecutando:

```
Bash
```

```
cat backup_documind_YYYYMMDD.sql | docker exec -i documind_db  
psql -U postgres -d documind_db
```

3. Descomprimir el archivo **tar.gz** de almacenamiento en la ruta **backend/storage/**.
4. Reiniciar el servicio backend.

9. Plan Básico de Mantenimiento y Monitoreo

9.1 Rutina Semanal de Depuración

- **Limpieza de registros fallidos:** Identificar documentos en estado **"Error"** mediante la consulta:

```
SQL  
SELECT id, filename, error_log FROM documents WHERE status =  
'Error';
```

- **Auditoría de integridad física:** Comparar los nombres de archivo presentes en **storage/** frente a los registros de la tabla **documents** para remover archivos huérfanos.

9.2 Rutina Mensual de Optimización y Rendimiento

- **Reorganización de índices vectoriales:** Ejecutar en la base de datos para mantener baja la latencia en las búsquedas RAG:

```
SQL  
VACUUM ANALYZE document_chunks;  
REINDEX TABLE document_chunks;
```

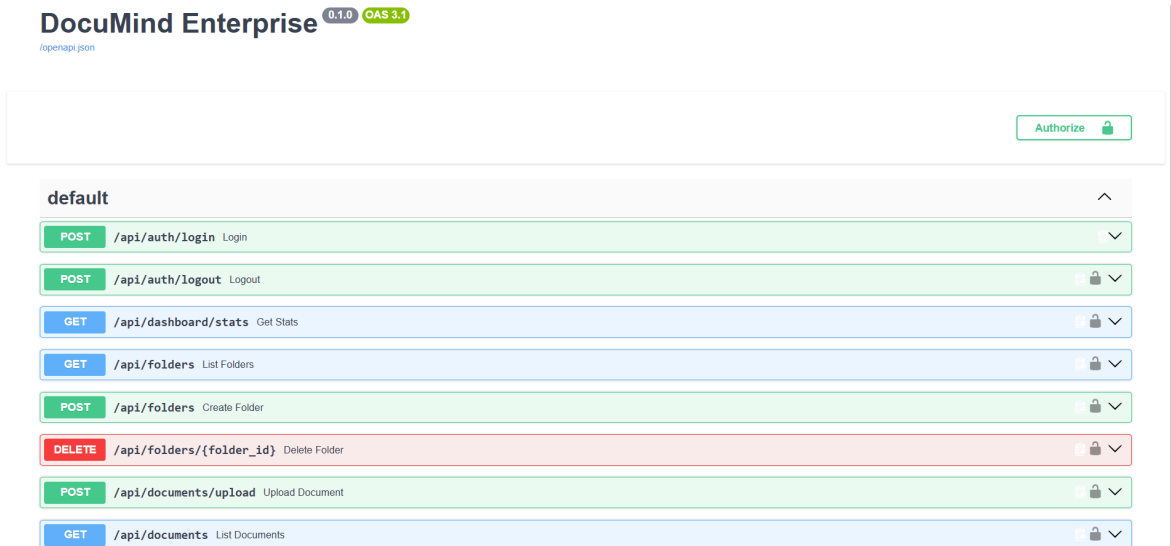
- **Depuración de imágenes de Docker:** Liberar espacio en disco eliminando contenedores temporales y volúmenes no referenciados:

```
Bash  
docker system prune -f
```

10. Evidencias del Sistema en Operación

A continuación se relacionan los puntos de verificación donde se demuestra el funcionamiento pleno e integrado de la plataforma para su presentación y sustentación ante el docente:

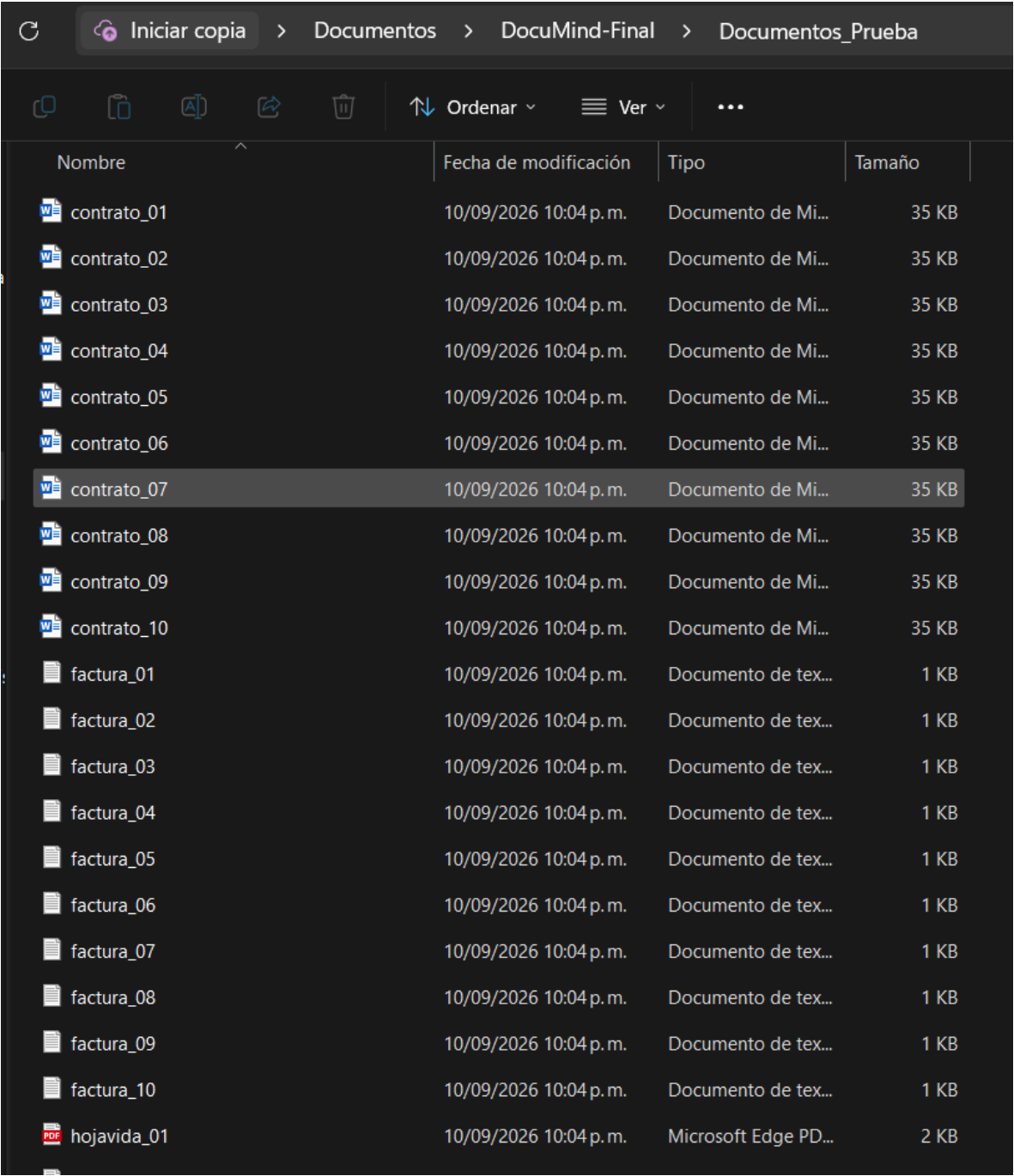
1. **Documentación Interactiva de la API (Swagger UI):** Accediendo a <http://localhost:8000/docs>, se verifica la disponibilidad y respuesta de los endpoints de autenticación, ingesta multiformato, consultas RAG y métricas.



2. **Operatividad del Contenedor de Base de Datos:** Verificación mediante el comando `docker ps` que demuestra el contenedor `documind_db` con estado saludable (*Up*) y el puerto 5432 mapeado.

```
PS C:\Users\USUARIO\Documents\DocuMind-Final> docker-compose up -d
>>
WARN[0000] C:\Users\USUARIO\Documents\DocuMind-Final\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 1/1
✓ Container documind_db Running
PS C:\Users\USUARIO\Documents\DocuMind-Final> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
fa1c6e9df85   pgvector/pgvector:pg16             "docker-entrypoint.s..." 30 hours ago  Up 2 hours   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp  documind_db
```

3. **Persistencia del Corpus de Prueba:** Directorio `backend/storage/` con los 30 archivos autogenerados de prueba y la tabla `documents` poblada con sus respectivos metadatos, resúmenes e índices vectoriales.



Repositorio

Nueva Carpeta

Cargar Archivo

Buscar por nombre...

Todas las Categorías

Todos los Estados

dd/mm/aaaa

Nombre	Categoría	Estado	Acciones
hojavirus_01.pdf	Sin Clasificar	Procesando	
factura_01.txt	Facturas y Cuentas de Cobro	Procesado	
contrato_01.docx	Contratos y Acuerdos Legales	Procesado	

Iniciar copia

Documentos

DocuMind-Final

backend

storage

Ordenar

Ver

Nombre	Fecha de modificación	Tipo	Tamaño
contrato_01	10/09/2026 10:14 p. m.	Documento de Mi...	35 KB
factura_01	10/09/2026 10:15 p. m.	Documento de tex...	1 KB
hojavirus_01	10/09/2026 10:15 p. m.	Microsoft Edge PD...	2 KB

documind=# SELECT id, filename, category, status FROM documents;

id	filename	category	status
21	contrato_01.docx	Contratos y Acuerdos Legales	Procesado
22	factura_01.txt	Facturas y Cuentas de Cobro	Procesado
23	hojavirus_01.pdf	Hojas de Vida / Perfiles Laborales	Procesado

(3 rows)

- Respuestas del Asistente IA (RAG):** Consultas en lenguaje natural respondidas satisfactoriamente en el frontend citando el archivo de procedencia correspondiente.

Consultas Semánticas (RAG)

¿Quién es el contratista y cuál es el valor del Contrato 01?

De acuerdo con el archivo **contrato_01.docx**: * **Contratista:** Andres Felipe * **Valor del contrato:** \$ 9.000.000 COP

Fuentes: contrato_01.docx

Ej: ¿Qué experiencia tiene Camila Rojas?

Enviar